Reviewer Pack — Minimal Alexander-Orlik-Solomon Complexity and SAT Clause De…

Entry 38f4ca4f6b69 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 05:11:31 UTC

Minimal Alexander-Orlik-Solomon Complexity and SAT Clause Depth Inequality

Entry ID: `38f4ca4f6b69` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-08 05:11:31 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Alexander Theory **Field B** (complexity object): Boolean Satisfiability (Clause Complexity)

Statement:

> For every Boolean formula φ, the minimal Alexander-Orlik-Solomon complexity (AOS(φ)) of its associated matroid is strictly greater than its clause depth (CD(φ)), such that AOS(φ) = Ω(CD(φ)).

Rationale (proposer's reasoning):

> Combinatorial Alexander theory provides a rich structure for analyzing the geometric and topological properties of graphs, while clause depth has been widely used as a measure of complexity in Boolean satisfiability problems. The conjecture links these two fields by suggesting an exponential gap between AOS complexity and clause depth, which could expose a fundamental separation in the hardness of certain SAT instances.

Taxonomy category: `combinatorial_alexander_theory` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `1ef6715649fa6576`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all random CNF formulas with clause depths ranging from 1 to 40, the Alexander-Orlik-Solomon complexity (AOS) of their associated matroids is strictly greater than their clause depth (CD), and this condition holds true for at least 95% of the generated formulas.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `'Alexander-Orlik-Solomon complexity' AND 'Boolean satisfiability' AND 'matroid'` - `'combinatorial Alexander theory' AND 'SAT clause depth inequality' AND 'Boolean formula'` - `AOS(φ) AND Ω(CD(φ)) AND matroid Boolean satisfiability`

Top relevant hits considered: - [http://arxiv.org/abs/1407.6169v3] Multiplicative Complexity of Vector Valued Boolean Functions - [http://arxiv.org/abs/1810.08668v2] Parity Decision Tree Complexity is Greater Than Granularity - [http://arxiv.org/abs/2401.14623v1] Structure in Communication Complexity and Constant-Cost Complexity Classes - [http://arxiv.org/abs/1004.0653v2] Exact Ramsey Theory: Green-Tao numbers and SAT - [http://arxiv.org/abs/math/9806161v1] Chern Classes in Alexander-Spanier Cohomology - [http://arxiv.org/abs/0706.2499v1] Alexander polynomials: Essential variables and multiplicities - [http://arxiv.org/abs/1406.4712v3] An algorithm for Boolean satisfiability based on generalized orthonormal expansion - [http://arxiv.org/abs/2305.10602v2] Density of smooth functions in Musielak-Orlicz-Sobolev spaces $W^{k,\Phi}(\Omega)$

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.2s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n, m):
        clauses = []
        for _ in range(m):
            clause = [random.randint(1, n), -random.randint(1, n)]
            while len(set(clause)) != 2:
                clause = [random.randint(1, n), -random.randint(1, n)]
            clauses.append(clause)
        return clauses

    def compute_clause_depth(cnf):
        return max(len(clause) for clause in cnf)

    def matroid_rank(matroid):
        rank = 0
        for i in range(len(matroid)):
            if all(all(j not in matroid[j] for j in matroid[i]) for i in range(i+1, len(matroid))):
                rank += 1
        return rank
```

```
    def alexander_orlik_solomon_complexity(matroid):
        n = len(matroid)
        AOS = [0] * (n + 1)
        AOS[0] = 1
        for i in range(1, n + 1):
            AOS[i] = sum(AOS[j] * (-1) ** (i - j) for j in range(i))
        return abs(sum(AOS[i] * matroid_rank(matroid[:i]) for i in range(n + 1)))

    def compute_metric(cnf):
        n = len(cnf)
        m = len(cnf[0])
        clause_depth = compute_clause_depth(cnf)
        matroid = [set(clause) for clause in cnf]
        aos_complexity = alexander_orlik_solomon_complexity(matroid)
        return aos_complexity, clause_depth

    n_max = 40
    instances_tested = 30
    metric_values = []

    for _ in range(instances_tested):
        n = random.randint(5, n_max)
        m = random.randint(n, n * n)
        cnf = generate_cnf(n, m)
        aos_complexity, clause_depth = compute_metric(cnf)
        if aos_complexity <= clause_depth:
            return {
                "metric_name": "AOS vs CD",
                "metric_value": 0.0,
                "instances_tested": instances_tested,
                "n_max": n_max,
                "conjecture_holds": False,
                "counterexample": f"Clause depth {clause_depth} >= AOS complexity {aos_complexity}
            }
        metric_values.append(aos_complexity / clause_depth)

    return {
        "metric_name": "AOS vs CD",
        "metric_value": sum(metric_values) / len(metric_values),
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)
```

```
    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(result
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.95:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"Clause depth >= AOS complexity\" first_failing_
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b85b5941.py", line 92, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b85b5941.py", line 64, in run_trial
    aos_complexity, clause_depth = compute_metric(cnf)
                                   ^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b85b5941.py", line 53, in compute_metri
    aos_complexity = alexander_orlik_solomon_complexity(matroid)
                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b85b5941.py", line 46, in alexander_orl
    return abs(sum(AOS[i] * matroid_rank(matroid[:i]) for i in range(n + 1)))
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b85b5941.py", line 46, in <genexpr>
    return abs(sum(AOS[i] * matroid_rank(matroid[:i]) for i in range(n + 1)))
                            ^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b85b5941.py", line 36, in matroid_rank
    if all(all(j not in matroid[j] for j in matroid[i]) for i in range(i+1, len(matroid))):
     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b85b5941.py", line 36, in <genexpr>
    if all(all(j not in matroid[j] for j in matroid[i]) for i in range(i+1, len(matroid))):
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b85b5941.py", line 36, in <genexpr>
    if all(all(j not in matroid[j] for j in matroid[i]) for i in range(i+1, len(matroid))):
                       ~~~~~~~^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means it did not complete the required computations to verify the conjecture. | next: Investigate and fix the crash in the test code to ensure it can run to completion and provide results.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14418
2	preregistration	ollama_remote	glm4:latest	0	0	12779
3	novelty	ollama_remote	glm4:latest	0	0	8606
4	novelty	ollama_remote	glm4:latest	0	0	10489
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	28786
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7155
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10746
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11257
9	judge	ollama_remote	glm4:latest	0	0	11576

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 115813 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/38f4ca4f6b69.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/38f4ca4f6b69.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/38f4ca4f6b69.tar.gz` (if generated)